



INTERVIEW PROGRAMMING PROBLEMS — CLUSTER 1



EASY LEVEL (L1 – Syntax & Memory Recall)

1 Print & Variables

Problem:

Print:

Hello, <name>. Welcome to Python.

Sample Input

Harish

Sample Output

Hello, Harish. Welcome to Python.

2 Even or Odd

Sample Input

7

Sample Output

Odd

3 Largest of Two Numbers

Sample Input

10 20

Sample Output

20

4 Sum of Numbers in a List

Sample Input

[1, 2, 3, 4]

Sample Output

10

5 Count Characters in a String (excluding spaces)

Sample Input

hello world

Sample Output

10

6 Reverse a String

Sample Input

python

Sample Output

nohtyp

7 Count Vowels

Sample Input

automation

Sample Output

6

8 Check Palindrome

Sample Input

madam

Sample Output

Palindrome

9 Print Numbers 1 to N

Sample Input: 5

Sample Output: 1 2 3 4 5

10 Factorial of a Number

Sample Input: 5

Sample Output: 120

1 1 Find Maximum in a List

Sample Input
[4, 8, 2, 10, 6]
Sample Output: 10

1 2 Count Occurrences

Sample Input
List: [1, 2, 2, 3, 2]
Element: 2
Sample Output: 3

1 3 Swap Two Variables

Sample Input: a = 5, b = 10
Sample Output: a = 10, b = 5

1 4 Remove Duplicates

Sample Input: [1, 2, 2, 3, 4, 4]
Sample Output: [1, 2, 3, 4]

1 5 Simple Calculator

Sample Input
Operation: +
Numbers: 5 3

Sample Output: 8

1 6 Check Leap Year

Sample Input: 2024
Sample Output: Leap Year

1 7 Print Pattern

Sample Input: 4
Sample Output
*
**

1 8 Count Words in a Sentence

Sample Input: Python is easy

Sample Output: 3

1 9 Find Common Elements

Sample Input

List1: [1, 2, 3]

List2: [2, 3, 4]

Sample Output

[2, 3]

2 0 Sum of Digits

Sample Input : 1234

Sample Output: 10



HOW TO PRACTICE CLUSTER 1 (VERY IMPORTANT)

Use this **exact loop** while practicing:

- 1** Read problem
- 2** Predict output mentally
- 3** Say logic aloud
- 4** Write code
- 5** Compare with sample output

If your output matches → **move on immediately**.

INTERVIEW PROGRAMMING PROBLEMS — CLUSTER 2

MEDIUM LEVEL

(Conceptual + Foundational Programming)

 Target: Crack L1 interviews for Tier-2 companies

These problems test:

- Logical thinking (not just syntax)
 - Data structure usage
 - Edge-case awareness
 - Ability to explain reasoning
-

1 Find Second Largest Element

Sample Input

[10, 20, 4, 45, 99]

Sample Output

45

2 Check Anagram

Check if two strings are anagrams.

Sample Input

listen
silent

Sample Output

Anagram

3 Count Frequency of Elements

Return frequency of each element in a list.

Sample Input

```
[1, 2, 2, 3, 1]
```

Sample Output

```
{1: 2, 2: 2, 3: 1}
```

4 Find Missing Number

Given numbers from 1 to n with one missing.

Sample Input

```
[1, 2, 4, 5]
```

Sample Output

```
3
```

5 Check Prime Number

Sample Input

```
29
```

Sample Output

```
Prime
```

6 Remove All Occurrences of an Element

Sample Input

```
List: [1, 2, 3, 2, 4]  
Element: 2
```

Sample Output

```
[1, 3, 4]
```

7 Reverse Words in a Sentence

Sample Input

```
Python is powerful
```

Sample Output

powerful is Python

8 Find First Non-Repeating Character

Sample Input

swiss

Sample Output

w

9 Fibonacci Series up to N

Sample Input

7

Sample Output

0 1 1 2 3 5 8

10 Check Balanced Parentheses

Sample Input

{ [()] }

Sample Output

Balanced

1 1 Merge Two Sorted Lists

Sample Input

[1, 3, 5]
[2, 4, 6]

Sample Output

[1, 2, 3, 4, 5, 6]

1 2 Find Duplicate Elements

Sample Input

[1,2,3,2,4,1]

Sample Output

[1,2]

1 3 Check Armstrong Number

Sample Input

153

Sample Output

Armstrong

1 4 Rotate List Right by K

Sample Input

List: [1,2,3,4,5]
K: 2

Sample Output

[4,5,1,2,3]

1 5 Count Uppercase & Lowercase Letters

Sample Input

PyThOn

Sample Output

Uppercase: 3, Lowercase: 3

1 6 Check Substring

Sample Input

String: automation
Substring: auto

Sample Output

1 7 Find Longest Word in Sentence

Sample Input

```
Python makes automation easier
```

Sample Output

```
automation
```

1 8 Convert List to Dictionary (Index as Key)

Sample Input

```
["a", "b", "c"]
```

Sample Output

```
{0: 'a', 1: 'b', 2: 'c'}
```

1 9 Sum of Even & Odd Numbers Separately

Sample Input

```
[1,2,3,4,5]
```

Sample Output

```
Even Sum: 6  
Odd Sum: 9
```

2 0 Check If List is Sorted

Sample Input

```
[1,2,3,4]
```

Sample Output

```
Sorted
```

PRACTICAL INTERVIEW TIPS — MEDIUM LEVEL

✓ What interviewers observe at this level

- Do you **break problem into steps**
 - Can you **choose correct data structure**
 - Do you consider **edge cases**
 - Can you **explain logic while coding**
-

✓ Golden approach to MEDIUM problems

Always speak this flow:

- 1 “I’ll understand the input and output”
 - 2 “I’ll choose a data structure”
 - 3 “I’ll write a basic solution first”
 - 4 “Then I’ll optimize if needed”
-

⚠ Common mistakes to avoid

- ✗ Jumping straight to code
 - ✗ Using advanced tricks too early
 - ✗ Ignoring edge cases (empty list, single element)
 - ✗ Staying silent while thinking
-

INTERVIEW PROGRAMMING PROBLEMS — CLUSTER 3

LARGE LEVEL

(OOP-Oriented, Real-World Design + Coding)

 *Target: Crack LI interviews for Tier-1 companies (EPAM, IDFC, similar)*

These problems test:

- Object-oriented thinking
- Class design & responsibility separation
- Ability to reason before coding
- Writing maintainable code, not hacks

 Interviewers here care **more about design clarity than full implementation.**

Bank Account System

Design a system with:

- BankAccount (parent)
- SavingsAccount, CurrentAccount (child)

Requirements

- Deposit
- Withdraw
- Interest calculation (differs per account)

Expected Behavior

```
SavingsAccount.calculate_interest() → specific logic  
CurrentAccount.calculate_interest() → different logic
```

Employee Payroll System

Design:

- Employee (abstract)
- FullTimeEmployee, ContractEmployee

Requirements

- Calculate salary
- Optional bonus

Expected Behavior

```
employee.calculate_salary()
```

(polymorphism)

3 Library Management System

Design classes for:

- Book
- Member
- Library

Operations

- Issue book
 - Return book
 - Check availability
-

4 Vehicle Rental System

Design:

- Vehicle (parent)
- Car, Bike (child)

Requirements

- Rent
 - Calculate rent per day
 - Different pricing logic
-

5 Shopping Cart System

Design:

- Product
- Cart
- Order

Operations

- Add/remove product
 - Calculate total price
 - Apply discount
-

6 Payment Processing System

Design:

- Payment (abstract)
- UPI, CreditCard, Wallet

Requirement

```
payment.process_payment()
```

Different behavior per type.

7 Notification System

Design:

- Notification (parent)
- EmailNotification
- SMSNotification

Requirement

```
notify()
```

8 Logger Utility (Singleton Pattern)

Design a logger class that:

- Allows only one instance
- Writes logs to file

Expected Behavior

```
logger1 is logger2 → True
```

9 File Processor

Design a system to:

- Read file
- Process content
- Write output

Use **composition**, not inheritance.

10 Online Order System

Design:

- Order
- Customer
- Item

Requirement

- Calculate total
 - Track order status
-

1 1 Test Automation Framework Skeleton

Design:

- BaseTest
- TestCase
- DriverManager

Explain:

- What goes in BaseTest
 - Why composition is preferred
-

1 2 ATM Machine

Design:

- ATM
- Account
- Transaction

Operations

- Withdraw
- Check balance

1 3 Shape Area Calculator

Design:

- Shape (abstract)
- Circle, Rectangle, Triangle

Requirement

```
calculate_area()
```

1 4 Configuration Reader

Design a class to:

- Read config file
- Provide values safely

Use:

- Encapsulation
 - Exception handling
-

1 5 Online Exam System

Design:

- Question
- Exam
- Student

Operations

- Start exam
 - Submit answers
 - Calculate score
-

1 6 Hotel Booking System

Design:

- Room
- Booking
- Customer

Requirement

- Book room
 - Cancel booking
-

1 7 File Upload Validator

Design a validator for:

- File type
- File size

Use:

- Custom exceptions
-

1 8 Browser Manager (Automation Context)

Design:

- BrowserManager
- BrowserFactory

Explain:

- Why factory pattern
 - How browser selection works
-

1 9 Inventory Management System

Design:

- Product
- Inventory

Operations

- Add stock
 - Reduce stock
 - Check availability
-

2 0 User Authentication System

Design:

- User
- AuthService

Operations

- Login
 - Logout
 - Validate credentials
-

PRACTICAL INTERVIEW TIPS — LARGE LEVEL (VERY IMPORTANT)

What interviewers really evaluate here

- Can you **identify entities**
 - Can you justify **inheritance vs composition**
 - Can you explain **why something is abstract**
 - Can you speak **before coding**
-

Golden OOP Answer Structure (MEMORIZE)

Always answer like this:

- 1** “Let me identify the entities”
- 2** “I’ll define common behavior in the parent”
- 3** “Variable behavior will be overridden”
- 4** “I’ll prefer composition where possible”

This structure alone scores points.

Common reasons candidates fail at this level

- ✗ Jump straight into coding
 - ✗ Over-engineering everything
 - ✗ Using inheritance everywhere
 - ✗ Not explaining design choices
-

Mental Anchor for LARGE level

“They are hiring how I think, not how much code I write.”

Alright. This is the **final, toughest cluster**.

Treat this as **L1 + L2 insurance**. If you're comfortable *thinking* through these, Tier-1 interviews stop feeling scary.

INTERVIEW PROGRAMMING PROBLEMS — CLUSTER 4

ADVANCED LEVEL

(L2-Ready · Design · Optimization · Patterns)

 *Goal: Ensure L1 success and strongly prepare for L2*

These problems test:

- Depth of thinking
- Design trade-offs
- Clean abstractions
- Scalability & extensibility
- How you reason under ambiguity

 **You are NOT expected to fully code all of these in interviews.**
You ARE expected to **structure + justify decisions**.

Design a Rate Limiter

Design a system that:

- Limits API requests per user
- Resets after a time window

Interview focus

- Data structures
 - Time-based logic
 - Scalability discussion
-

LRU Cache Implementation

Design an LRU Cache with:

- `get(key)`
- `put(key, value)`

Constraints: $O(1)$ operations

3 Parking Lot System

Design a parking lot with:

- Multiple vehicle types
- Slot allocation
- Entry & exit tracking

Interview focus

- OOP hierarchy
 - Responsibility separation
-

4 Thread-Safe Singleton Logger

Design a logger that:

- Has only one instance
- Is safe in multithreaded environments

Interview focus

- Singleton pattern
 - Concurrency awareness
-

5 URL Shortener (TinyURL)

Design a system that:

- Converts long URL → short URL
- Retrieves original URL

Interview focus

- Mapping strategy
 - Collision handling
-

6 File Processing Pipeline

Design a pipeline to:

- Read large files
- Process line by line

- Write results safely

Interview focus

- Memory efficiency
 - Streaming approach
-

7 Notification System (Extensible)

Design a system where: New notification types can be added without changing existing code

Interview focus

- Open–Closed Principle
 - Strategy pattern
-

8 Test Automation Framework (Advanced)

Design a scalable automation framework:

- Parallel execution
- Reporting
- Retry logic

Interview focus:

- Architecture, not syntax
-

9 Online Voting System

Design:

- Vote casting
- Result counting
- Prevent duplicate voting

Interview focus

- Data integrity
 - Validation logic
-

10 Rule Engine

Design a rule engine where:

- Rules can be added dynamically
- No code change needed

Interview focus

- Configuration-driven design
 - Decoupling logic
-

1 1 Bank Transaction Processor

Design a system that:

- Processes deposits & withdrawals
- Ensures atomicity

Interview focus

- Error handling
 - Consistency guarantees
-

1 2 Object Pool Pattern

Design an object pool for:

- Database connections / browser drivers

Interview focus

- Resource reuse
 - Lifecycle management
-

1 3 Payment Gateway Router

Design a router that:

- Selects payment provider based on conditions
- Allows easy addition of providers

Interview focus: Factory + Strategy patterns

1 4 Versioned Configuration Manager

Design a system that:

- Maintains config versions
- Allows rollback

Interview focus

- State management
 - Encapsulation
-

1 5 Search Autocomplete System

Design: Prefix-based suggestions

Interview focus

- Data structures (Trie discussion)
 - Performance
-

1 6 Job Scheduler

Design a scheduler that:

- Runs jobs at fixed intervals
- Handles failures

Interview focus

- Time handling
 - Retry strategy
-

1 7 Access Control System

Design:

- Roles
- Permissions
- Authorization checks

Interview focus

- RBAC concepts
 - Extensibility
-

1 8 Multi-Tenant Application Design

Design an app that:

- Serves multiple clients
- Keeps data isolated

Interview focus

- Design clarity
 - Separation concerns
-

1 9 Metrics & Monitoring System

Design:

- Metric collection
- Aggregation
- Reporting

Interview focus

- Observer pattern
 - Decoupling
-

2 0 Failover Mechanism

Design a system that:

- Switches to backup on failure
- Minimizes downtime

Interview focus

- Resilience thinking
 - Graceful degradation
-

PRACTICAL INTERVIEW TIPS — ADVANCED LEVEL (VERY IMPORTANT)

✓ What interviewers look for here

- Do you **ask clarifying questions**
 - Can you **define assumptions**
 - Do you think in **components**
 - Can you say **why**, not just **how**
-

✓ Golden Answer Framework (MEMORIZE THIS)

Whenever an advanced problem is asked, speak in this order:

- 1 “Let me clarify assumptions”
- 2 “I’ll identify core components”
- 3 “I’ll define responsibilities”
- 4 “I’ll discuss trade-offs”
- 5 “I’ll mention scalability improvements”

Even without full code, this scores highly.

⚠ Common mistakes at ADVANCED level

- ✗ Trying to code everything
 - ✗ Not asking clarifying questions
 - ✗ Over-optimizing too early
 - ✗ Ignoring edge cases & failure paths
-

Mental Anchor for ADVANCED level

“They don’t expect perfection. They expect maturity.”
